

Entregable 7. Evidencias de los tres casos del punto 7

Integrantes: Derly Dayana Garcia Carrillo · Orlando Rojas Narvaez | Docente: Juan Carlos Polania Cortes | Periodo 2026-2
| Repositorio: github.com/OrlandoRojasNa/taller02-rabbitmq-docker

Caso 1: consumidor detenido

Con `docker compose stop consumidor` se publicaron 5 mensajes. Se acumularon en la cola y, al ejecutar `docker compose start consumidor`, se consumieron todos.

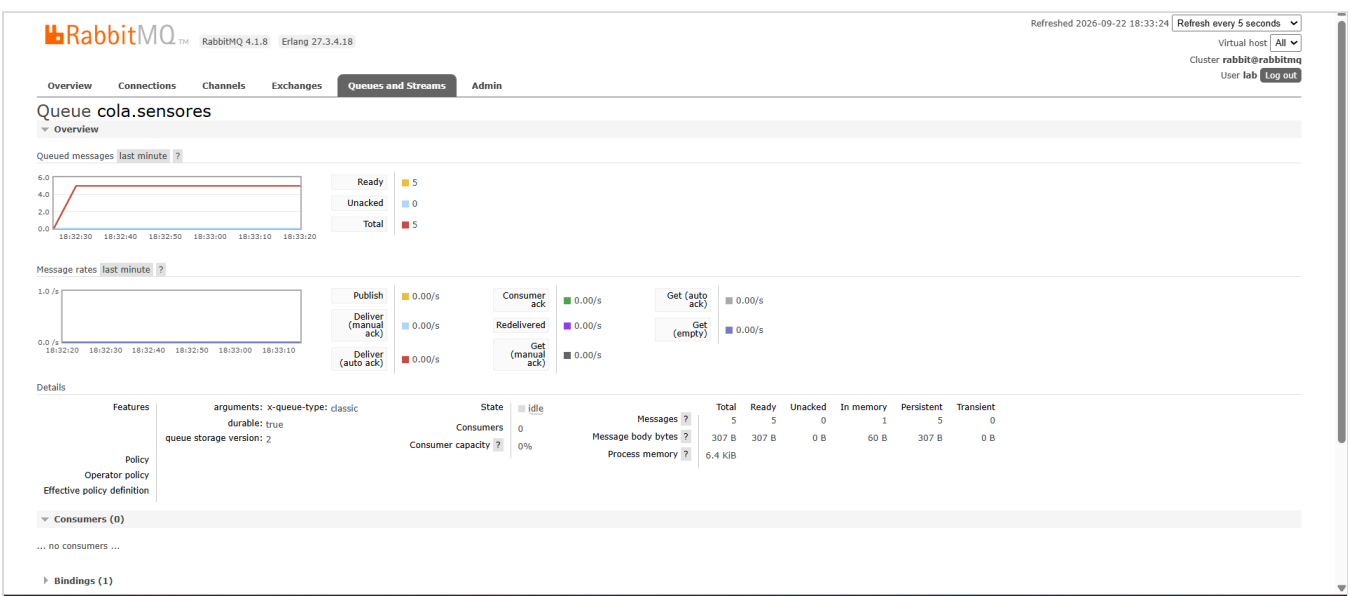


Figura 7. Antes. Consumidor detenido: **Ready = 5, Consumers (0)**. La cola es `durable: true` y los 5 mensajes son **Persistent**.

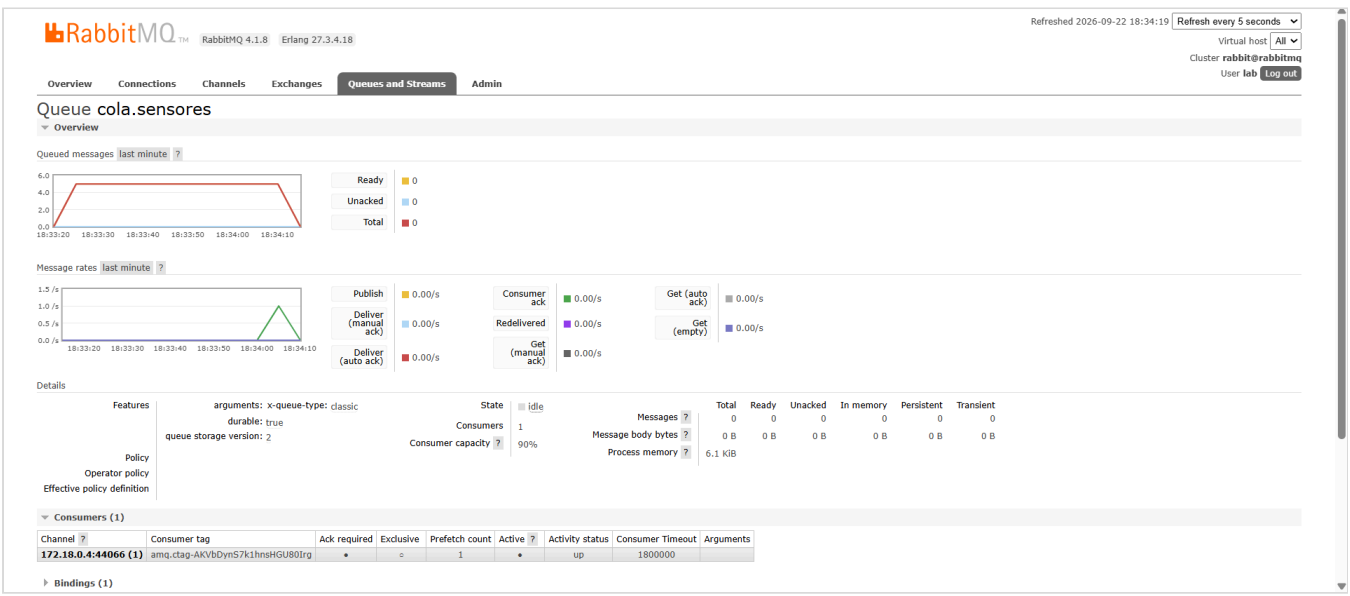


Figura 8. Después. Consumidor levantado: la gráfica baja de 5 a 0, pico de *Consumer ack*, **Consumers (1)** con **Ack required** (ACK manual) y **Prefetch count 1**.

```
PS C:\cc-repos\taller02-rabbitmq-docker> docker compose logs consumidor | Select-String "Mensaje recibido" | Select-Object -Last 5

consumidor | 2026-09-22 18:34:08.845 INFO Mensajelister - [#1] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.845 -05:00 | id: a7a9c8a3-93e4-471e-83c7-7acf109f69f5
| reentregado: false | contenido: {"sensor":"00-01","valor":1,"unidad":"mg/L","iteracion":232}
consumidor | 2026-09-22 18:34:08.852 INFO Mensajelister - [#2] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.852 -05:00 | id: b870f8a5-c217-4aa9-8726-ac09cb5ba783
| reentregado: false | contenido: {"sensor":"00-01","valor":874,"unidad":"mg/L","iteracion":268}
consumidor | 2026-09-22 18:34:08.854 INFO Mensajelister - [#3] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.854 -05:00 | id: fba42fd8-9952-4bd0-98c0-0cc00287bf31
| reentregado: false | contenido: {"sensor":"00-01","valor":246,"unidad":"mg/L","iteracion":683}
consumidor | 2026-09-22 18:34:08.856 INFO Mensajelister - [#4] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.856 -05:00 | id: cf7ba48f-8bce-4b75-90c2-293f5d456dc0
| reentregado: false | contenido: {"sensor":"00-01","valor":645,"unidad":"mg/L","iteracion":488}
consumidor | 2026-09-22 18:34:08.858 INFO Mensajelister - [#5] Mensaje recibido | hora de recepcion: 2026-09-22 18:34:08.858 -05:00 | id: 02f33544-aa07-4c06-ae7d-70896f19a691
| reentregado: false | contenido: {"sensor":"00-01","valor":12,"unidad":"mg/L","iteracion":210}
```

Figura 9. Log del consumidor: los 5 mensajes acumulados, recibidos a las 18:34:08, apenas se levantó el consumidor.

Caso 2: reinicio de RabbitMQ

Con el consumidor detenido (`docker compose stop consumidor`) se publicaron varios mensajes desde Postman y se anotó el valor de *Ready*. Luego se reinició el broker con `docker compose restart rabbitmq`, se volvió a iniciar sesión en la consola y se comprobó que la cola tenía **la misma cantidad** de mensajes. Por último, `docker compose start consumidor` los procesó todos.

Los mensajes sobreviven porque se cumplen las cuatro condiciones a la vez:

Condición	Dónde se configura
La cola es durable (su definición se guarda en disco)	<code>rabbitmq/definitions.json</code> → <code>"durable": true</code>
Cada mensaje es persistente (se escribe en disco)	<code>PublicadorService</code> → <code>MessageDeliveryMode.PERSISTENT</code>
Los datos del nodo viven fuera del contenedor	<code>docker-compose.yml</code> → volumen <code>rabbitmq_data:/var/lib/rabbitmq</code>
El nombre del nodo no cambia entre reinicios	<code>docker-compose.yml</code> → <code>hostname: rabbitmq</code>

Además, el productor no necesita reiniciarse: Spring AMQP abre una conexión nueva en la siguiente publicación, y el consumidor reintenta conectarse cada 5 s.

Captura no incluida en esta entrega: 08-caso2-antes.png

Figura 10. Antes. Consumidor detenido, mensajes acumulados en `cola.sensores` (*Ready* > 0, **Consumers (0)**), justo antes de `docker compose restart rabbitmq`.

Captura no incluida en esta entrega: 08-caso2-despues.png

Figura 11. Después. Tras el reinicio del broker (la gráfica se reinicia porque las estadísticas no se conservan), la cola sigue existiendo y *Ready* muestra **la misma cantidad** de mensajes: no se perdió ninguno.

Caso 3: mensaje inválido

Si la API recibe un cuerpo que no es JSON válido, responde con `status` en `false` y no publica nada en la cola. Se anotó el total de `cola.sensores`, se envió *Enviar mensaje invalido (JSON mal formado)* desde Postman y se comprobó que el total no cambió.

La validación ocurre **antes** de tocar RabbitMQ: `PublicadorService.validar()` exige un objeto JSON con al menos un campo y sin contenido extra al final (`FAIL_ON_TRAILING_TOKENS`). Si falla, lanza `MensajeInvalidoException`, `ManejadorErrores` la convierte en `400` con `{"status": false,`

"message": "Formato de mensaje inválido"} y en el log del productor queda Mensaje rechazado, no se publica: ...

Cuerpo enviado	HTTP	¿Se publica?
{"sensor": "0D-01", "valor": } (mal formado)	400	No
esto no es json (texto plano)	400	No
(cuerpo vacío)	400	No
{"sensor": "0D-01"} basura (contenido extra)	400	No
[1, 2, 3] o {} (no es un objeto con campos)	400	No

Captura no incluida en esta entrega: 09-caso3-antes.png

Figura 12. Antes. Total de mensajes de cola.sensores antes de enviar el mensaje inválido.

Captura no incluida en esta entrega: 09-caso3-despues.png

Figura 13. Después. Postman recibe 400 Bad Request con "status": false y el total de la cola no cambia.